

**Name: Monisha Raghini Chelle**

**RollNo:2520030537**

## **Section-6**

### **Practical Programs**

#### **Practical-7**

7. Write a program that prints the addresses of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.

#### **Program:**

```
#include <stdio.h>
#include <stdlib.h>
int global=100;
static int s=50;

int main()
{
    int stack=10;
    int *heap=(int*)malloc(sizeof(int));

    printf("Code Segment : %p\n",main);
    printf("Global Segment : %p\n", &global);
    printf("Static Segment : %p\n", &s);
    printf("Heap Segment : %p\n", heap);
    printf("Stack Segment : %p\n", &stack);
    free(heap);
    return 0;
}
```

Make use of the /proc//maps file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

~~PS~~

cat /proc/PID/maps

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/28-9-26
monis@DESKTOP-C3GM4T6:~$ ls
Linux-System-Programming-Week1  OSSP-Practical      fileio    hello    monis    sample1.c.save
OSLAB                          Simple-Commands-Week1 fileio.c   hello.c  sample1  student.txt
OSLAB14                        example.txt          foldername moni     sample1.c
monis@DESKTOP-C3GM4T6:~$ cd OSLAB
monis@DESKTOP-C3GM4T6:~/OSLAB$ ls
28-9-26  Scheduling-Algorithms  hello.c          hello_link  skill
OSLAB28  file1.txt              hello.c:Zone.Identifier  hello_shortcut  test1
Pipes    hello                  hello:Zone.Identifier  newfile.txt    virtual_address
monis@DESKTOP-C3GM4T6:~/OSLAB$ cd 28-9-26
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/28-9-26
monis@DESKTOP-C3GM4T6:~/OSLAB/28-9-26$ nano practical7.c
monis@DESKTOP-C3GM4T6:~/OSLAB/28-9-26$
```

Paste the Code

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/28-9-26
GNU nano 8.7.1 practical7.c *
#include <stdio.h>
#include <stdlib.h>

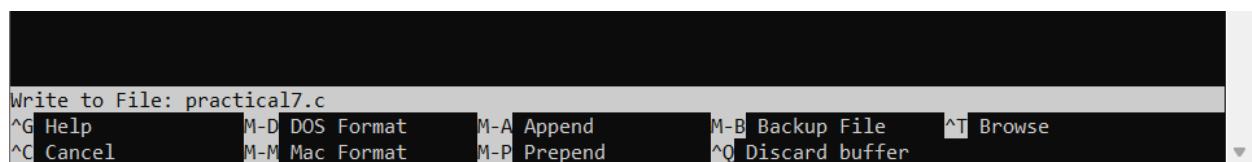
int global = 100;
static int s = 50;

int main()
{
    int stack = 10;
    int *heap = (int*)malloc(sizeof(int));

    printf("Code Segment : %p\n", main);
    printf("Global Segment : %p\n", &global);
    printf("Static Segment : %p\n", &s);
    printf("Heap Segment : %p\n", heap);
    printf("Stack Segment : %p\n", &stack);

    free(heap);
    return 0;
}
```

save it



run:

**gcc practical7a.c -o practical7a**

**./practical7a**

```
Select monis@DESKTOP-C3GM4T6: ~/OSLAB/28-9-26
monis@DESKTOP-C3GM4T6:~/OSLAB/28-9-26$ nano practical7.c
monis@DESKTOP-C3GM4T6:~/OSLAB/28-9-26$ gcc practical7.c -o practical7
monis@DESKTOP-C3GM4T6:~/OSLAB/28-9-26$ ./practical7
Code Segment : 0x5df29b5671a9
Global Segment : 0x5df29b56a010
Static Segment : 0x5df29b56a014
Heap Segment : 0x5df2d418f010
Stack Segment : 0x7ffe9ac4f30c
monis@DESKTOP-C3GM4T6:~/OSLAB/28-9-26$
```

**Commands given in Image (above) for memory analysis**

**ps**

**cat /proc/PID/maps**

```
monis@DESKTOP-C3GM4T6:~/OSLAB/28-9-26$ ps
```

```
PID TTY          TIME CMD
 4112 pts/0        00:00:00 bash
 4374 pts/0        00:00:00 ps
```

```
monis@DESKTOP-C3GM4T6:~/OSLAB/28-9-26$ cat /proc/4112/maps
```

```
5fa8dd38e000-5fa8dd3bf000 r--p 00000000 08:30 1502 /usr/bin/bash
5fa8dd3bf000-5fa8dd4c2000 r-xp 00031000 08:30 1502 /usr/bin/bash
5fa8dd4c2000-5fa8dd4f9000 r--p 00134000 08:30 1502 /usr/bin/bash
5fa8dd4f9000-5fa8dd4fd000 r--p 0016b000 08:30 1502 /usr/bin/bash
5fa8dd4fd000-5fa8dd506000 rw-p 0016f000 08:30 1502 /usr/bin/bash
5fa8dd506000-5fa8dd512000 rw-p 00000000 00:00 0
5fa915bb7000-5fa915d22000 rw-p 00000000 00:00 0 [heap]
7e52abe00000-7e52abe28000 r--p 00000000 08:30 34698 /usr/lib/x86_64-linux-gnu/libc.
so.6
7e52abe28000-7e52abfc0000 r-xp 00028000 08:30 34698 /usr/lib/x86_64-linux-gnu/libc.
so.6
7e52abfc0000-7e52ac00e000 r--p 001c0000 08:30 34698 /usr/lib/x86_64-linux-gnu/libc.
so.6
7e52ac00e000-7e52ac012000 r--p 0020e000 08:30 34698 /usr/lib/x86_64-linux-gnu/libc.
so.6
7e52ac012000-7e52ac014000 rw-p 00212000 08:30 34698 /usr/lib/x86_64-linux-gnu/libc.
so.6
7e52ac014000-7e52ac021000 rw-p 00000000 00:00 0
7e52ac033000-7e52ac08e000 r--p 00000000 08:30 13633 /usr/lib/locale/C.utf8/LC_CTYPE
7e52ac08e000-7e52ac08f000 r--p 00000000 08:30 13639 /usr/lib/locale/C.utf8/LC_NUMER
IC
7e52ac08f000-7e52ac090000 r--p 00000000 08:30 13643 /usr/lib/locale/C.utf8/LC_TIME
7e52ac090000-7e52ac091000 r--p 00000000 08:30 13632 /usr/lib/locale/C.utf8/LC_COLLA
TE
7e52ac091000-7e52ac092000 r--p 00000000 08:30 13637 /usr/lib/locale/C.utf8/LC_MONET
ARY
7e52ac092000-7e52ac093000 r--p 00000000 08:30 13636 /usr/lib/locale/C.utf8/LC_MESSA
GES/SYS_LC_MESSAGES
7e52ac093000-7e52ac09a000 r--s 00000000 08:30 34694 /usr/lib/x86_64-linux-gnu/gconv
/gconv-modules.cache
7e52ac09a000-7e52ac09d000 rw-p 00000000 00:00 0
7e52ac09d000-7e52ac0ab000 r--p 00000000 08:30 14513 /usr/lib/x86_64-linux-gnu/libti
nfo.so.6.6
7e52ac0ab000-7e52ac0bf000 r-xp 0000e000 08:30 14513 /usr/lib/x86_64-linux-gnu/libti
nfo.so.6.6
7e52ac0bf000-7e52ac0cd000 r--p 00022000 08:30 14513 /usr/lib/x86_64-linux-gnu/libti
nfo.so.6.6
7e52ac0cd000-7e52ac0d1000 r--p 0002f000 08:30 14513 /usr/lib/x86_64-linux-gnu/libti
nfo.so.6.6
7e52ac0d1000-7e52ac0d2000 rw-p 00033000 08:30 14513 /usr/lib/x86_64-linux-gnu/libti
nfo.so.6.6
7e52ac0d2000-7e52ac0d3000 r--p 00000000 08:30 13641 /usr/lib/locale/C.utf8/LC_PAPER
7e52ac0d3000-7e52ac0d4000 r--p 00000000 08:30 13638 /usr/lib/locale/C.utf8/LC_NAME
7e52ac0d4000-7e52ac0d5000 r--p 00000000 08:30 13631 /usr/lib/locale/C.utf8/LC_ADDRE
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/28-9-26
7e52ac08e000-7e52ac08f000 r--p 00000000 08:30 13639 /usr/lib/locale/C.utf8/LC_NUMER
IC
7e52ac08f000-7e52ac090000 r--p 00000000 08:30 13643 /usr/lib/locale/C.utf8/LC_TIME
7e52ac090000-7e52ac091000 r--p 00000000 08:30 13632 /usr/lib/locale/C.utf8/LC_COLLA
TE
7e52ac091000-7e52ac092000 r--p 00000000 08:30 13637 /usr/lib/locale/C.utf8/LC_MONET
ARY
7e52ac092000-7e52ac093000 r--p 00000000 08:30 13636 /usr/lib/locale/C.utf8/LC_MESSA
GES/SYS_LC_MESSAGES
7e52ac093000-7e52ac09a000 r--s 00000000 08:30 34694 /usr/lib/x86_64-linux-gnu/gconv
/gconv-modules.cache
7e52ac09a000-7e52ac09d000 rw-p 00000000 00:00 0
7e52ac09d000-7e52ac0ab000 r--p 00000000 08:30 14513 /usr/lib/x86_64-linux-gnu/libti
nfo.so.6.6
7e52ac0ab000-7e52ac0bf000 r-xp 0000e000 08:30 14513 /usr/lib/x86_64-linux-gnu/libti
nfo.so.6.6
7e52ac0bf000-7e52ac0cd000 r--p 00022000 08:30 14513 /usr/lib/x86_64-linux-gnu/libti
nfo.so.6.6
7e52ac0cd000-7e52ac0d1000 r--p 0002f000 08:30 14513 /usr/lib/x86_64-linux-gnu/libti
nfo.so.6.6
7e52ac0d1000-7e52ac0d2000 rw-p 00033000 08:30 14513 /usr/lib/x86_64-linux-gnu/libti
nfo.so.6.6
7e52ac0d2000-7e52ac0d3000 r--p 00000000 08:30 13641 /usr/lib/locale/C.utf8/LC_PAPER
7e52ac0d3000-7e52ac0d4000 r--p 00000000 08:30 13638 /usr/lib/locale/C.utf8/LC_NAME
7e52ac0d4000-7e52ac0d5000 r--p 00000000 08:30 13631 /usr/lib/locale/C.utf8/LC_ADDRE
SS
7e52ac0d5000-7e52ac0d6000 r--p 00000000 08:30 13642 /usr/lib/locale/C.utf8/LC_TELEP
HONE
7e52ac0d6000-7e52ac0d7000 r--p 00000000 08:30 13635 /usr/lib/locale/C.utf8/LC_MEASU
REMENT
7e52ac0d7000-7e52ac0d8000 r--p 00000000 08:30 13634 /usr/lib/locale/C.utf8/LC_IDENT
IFICATION
7e52ac0d8000-7e52ac0da000 rw-p 00000000 00:00 0
7e52ac0da000-7e52ac0db000 r--p 00000000 08:30 34695 /usr/lib/x86_64-linux-gnu/ld-li
nux-x86-64.so.2
7e52ac0db000-7e52ac10a000 r-xp 00001000 08:30 34695 /usr/lib/x86_64-linux-gnu/ld-li
nux-x86-64.so.2
7e52ac10a000-7e52ac115000 r--p 00030000 08:30 34695 /usr/lib/x86_64-linux-gnu/ld-li
nux-x86-64.so.2
7e52ac115000-7e52ac117000 r--p 0003b000 08:30 34695 /usr/lib/x86_64-linux-gnu/ld-li
nux-x86-64.so.2
7e52ac117000-7e52ac118000 rw-p 0003d000 08:30 34695 /usr/lib/x86_64-linux-gnu/ld-li
nux-x86-64.so.2
7e52ac118000-7e52ac119000 rw-p 00000000 00:00 0
7ffded3f5000-7ffded416000 rw-p 00000000 00:00 0 [stack]
7ffded520000-7ffded524000 r--p 00000000 00:00 0 [vvar]
7ffded524000-7ffded526000 r-xp 00000000 00:00 0 [vdso]
monis@DESKTOP-C3GM4T6:~/OSLAB/28-9-26$
```

## Practical-8a

8. Develop a program using `malloc()`, `calloc()`, `realloc()`, and `free()`. Monitor memory allocation behavior and identify memory leaks using Valgrind.

### Program:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *a,*b;

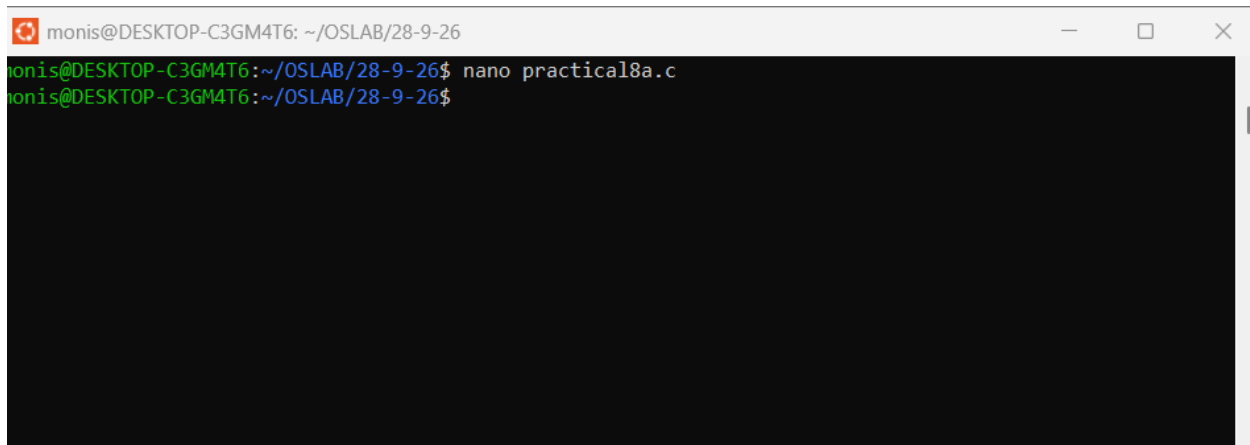
    a=(int*)malloc(5*sizeof(int));

    b=(int*)calloc(5,sizeof(int));

    a=(int*)realloc(a,10*sizeof(int));

    free(a);
    free(b);

    return 0;
}
```

A screenshot of a terminal window. The title bar shows the user 'monis' on a machine named 'DESKTOP-C3GM4T6' at the directory '~/OSLAB/28-9-26'. The terminal shows the command 'nano practical8a.c' being entered at the prompt. The background of the terminal is black, and the text is green. The window has standard Linux window controls (minimize, maximize, close) on the right side of the title bar.

Paste the code

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/28-9-26
GNU nano 8.7.1 practical8a.c *
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *a, *b;

    a = (int*)malloc(5 * sizeof(int));

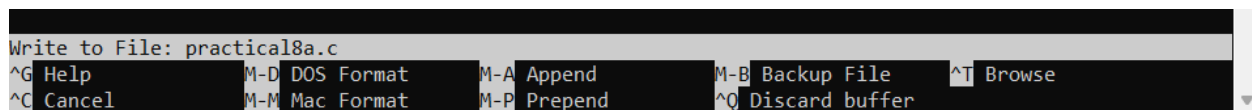
    b = (int*)calloc(5, sizeof(int));

    a = (int*)realloc(a, 10 * sizeof(int));

    free(a);
    free(b);

    return 0;
}
```

save it using Ctrl + O, Enter, then Ctrl + X.



**Compile and run**

**gcc -g practical8a.c -o practical8a**

**./practical8a**

-

**valgrind --leak-check=full ./practical8a**

```
Select monis@DESKTOP-C3GM4T6: ~/OSLAB/28-9-26
monis@DESKTOP-C3GM4T6:~/OSLAB/28-9-26$ nano practical8a.c
monis@DESKTOP-C3GM4T6:~/OSLAB/28-9-26$ gcc -g practical8a.c -o practical8a
monis@DESKTOP-C3GM4T6:~/OSLAB/28-9-26$ ./practical8a
monis@DESKTOP-C3GM4T6:~/OSLAB/28-9-26$ valgrind --leak-check=full ./practical8a
==4432== Memcheck, a memory error detector
==4432== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==4432== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==4432== Command: ./practical8a
==4432==
==4432==
==4432== HEAP SUMMARY:
==4432==   in use at exit: 0 bytes in 0 blocks
==4432==   total heap usage: 3 allocs, 3 frees, 80 bytes allocated
==4432==
==4432== All heap blocks were freed -- no leaks are possible
==4432==
==4432== For lists of detected and suppressed errors, rerun with: -s
==4432== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
monis@DESKTOP-C3GM4T6:~/OSLAB/28-9-26$
```

## Practical -8b

Run `valgrind` and check the result as: All heap blocks were freed -- no leaks are possible.

Develop a program that demonstrates Copy-on-Write behavior after a `fork()`. Analyze memory usage before and after modifying data in the child process.

```
#include <stdio.h>
#include <unistd.h>
int main()
{
    int x=100;
    if(fork()==0)
    {
        printf("Child Before=%d\n",x);
        x=500;
        printf("Child After=%d\n",x);
    }
    else
    {
        sleep(2);
        printf("Parent=%d\n",x);
    }

    return 0;
}
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/28-9-26
monis@DESKTOP-C3GM4T6:~/OSLAB/28-9-26$ nano practical8b.c
monis@DESKTOP-C3GM4T6:~/OSLAB/28-9-26$
```

Paste the code and save using Ctrl + O, Enter, then Ctrl + X.

```
GNU nano 8.7.1 practical8b.c *
#include <stdio.h>
#include <unistd.h>

int main()
{
    int x = 100;

    if (fork() == 0)
    {
        printf("Child Before=%d\n", x);
        x = 500;
        printf("Child After=%d\n", x);
    }
    else
    {
        sleep(2);
        printf("Parent=%d\n", x);
    }

    return 0;
}
```

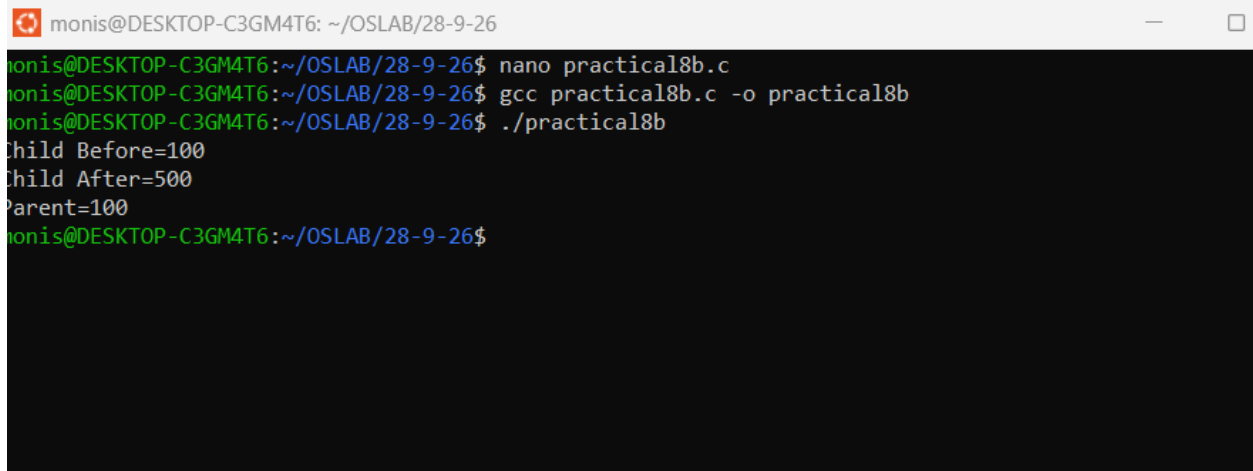
Write to File: practical8b.c

|           |                |             |                   |           |
|-----------|----------------|-------------|-------------------|-----------|
| ^G Help   | M-D DOS Format | M-A Append  | M-B Backup File   | ^T Browse |
| ^C Cancel | M-M Mac Format | M-P Prepend | ^O Discard buffer |           |

## Compile and run in Ubuntu

`gcc practical8b.c -o practical8b`

`./practical8b`



```
monis@DESKTOP-C3GM4T6: ~/OSLAB/28-9-26
monis@DESKTOP-C3GM4T6:~/OSLAB/28-9-26$ nano practical8b.c
monis@DESKTOP-C3GM4T6:~/OSLAB/28-9-26$ gcc practical8b.c -o practical8b
monis@DESKTOP-C3GM4T6:~/OSLAB/28-9-26$ ./practical8b
Child Before=100
Child After=500
Parent=100
monis@DESKTOP-C3GM4T6:~/OSLAB/28-9-26$
```